

Autodesk® Scaleform®

Scaleform

ActionScript

Scaleform 3.1

C++ Flash

Mustafa Thamer Prasad Silva
2.01
2010 9 21

Copyright Notice

Autodesk® Scaleform® 4.2

© 2012 Autodesk, Inc. All rights reserved. Except as otherwise permitted by Autodesk, Inc., this publication, or parts thereof, may not be reproduced in any form, by any method, for any purpose.

Certain materials included in this publication are reprinted with the permission of the copyright holder.

The following are registered trademarks or trademarks of Autodesk, Inc., and/or its subsidiaries and/or affiliates in the USA and other countries: 123D, 3ds Max, Algor, Alias, AliasStudio, ATC, AUGI, AutoCAD, AutoCAD Learning Assistance, AutoCAD LT, AutoCAD Simulator, AutoCAD SQL Extension, AutoCAD SQL Interface, Autodesk, Autodesk Homestyler, Autodesk Intent, Autodesk Inventor, Autodesk MapGuide, Autodesk Streamline, AutoLISP, AutoSketch, AutoSnap, AutoTrack, Backburner, Backdraft, Beast, Beast (design/logo) Built with ObjectARX (design/logo), Burn, Buzzsaw, CAiCE, CFdesign, Civil 3D, Cleaner, Cleaner Central, ClearScale, Colour Warper, Combustion, Communication Specification, Constructware, Content Explorer, Creative Bridge, Dancing Baby (image), DesignCenter, Design Doctor, Designer's Toolkit, DesignKids, DesignProf, DesignServer, DesignStudio, Design Web Format, Discreet, DWF, DWG, DWG (design/logo), DWG Extreme, DWG TrueConvert, DWG TrueView, DWFX, DXF, Ecotect, Evolver, Exposure, Extending the Design Team, Face Robot, FBX, Fempro, Fire, Flame, Flare, Flint, FMDesktop, Freewheel, GDX Driver, Green Building Studio, Heads-up Design, Heidi, Homestyler, HumanIK, i-drop, ImageModeler, iMOUT, Incinerator, Inferno, Instructables, Instructables (stylized robot design/logo), Inventor, Inventor LT, Kynapse, Kynogon, LandXplorer, Lustre, MatchMover, Maya, Mechanical Desktop, MIMI, Moldflow, Moldflow Plastics Advisers, Moldflow Plastics Insight, Moondust, MotionBuilder, Movimento, MPA, MPA (design/logo), MPI (design/logo), MPX, MPX (design/logo), Mudbox, Multi-Master Editing, Navisworks, ObjectARX, ObjectDBX, Opticore, Pipeplus, Pixlr, Pixlr-o-matic, PolarSnap, Powered with Autodesk Technology, Productstream, ProMaterials, RasterDWG, RealDWG, Real-time Roto, Recognize, Render Queue, Retimer, Reveal, Revit, RiverCAD, Robot, Scaleform, Scaleform GFx, Showcase, Show Me, ShowMotion, SketchBook, Smoke, Softimage, Sparks, SteeringWheels, Stitcher, Stone, StormNET, Tinkerbox, ToolClip, Topobase, Toxik, TrustedDWG, T-Splines, U-Vis, ViewCube, Visual, Visual LISP, Vtour, WaterNetworks, Wire, Wiretap, WiretapCentral, XSI.

All other brand names, product names or trademarks belong to their respective holders.

Disclaimer

THIS PUBLICATION AND THE INFORMATION CONTAINED HEREIN IS MADE AVAILABLE BY AUTODESK, INC. "AS IS." AUTODESK, INC. DISCLAIMS ALL WARRANTIES, EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE REGARDING THESE MATERIALS.

Autodesk Scaleform

Scaleform

Autodesk Scaleform Corporation

6305 Ivy Lane, Suite 310

Greenbelt, MD 20770, USA

www.scaleform.com

Email info@scaleform.com

+1 (301) 446-3200

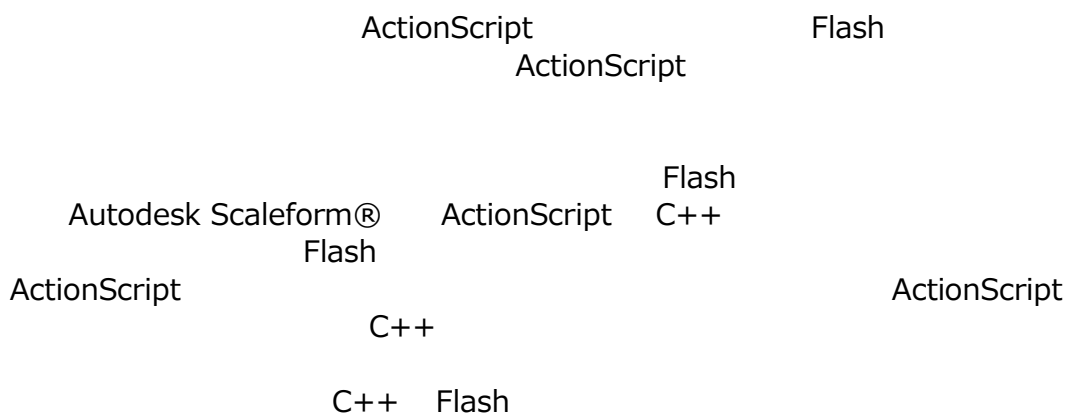
Fax +1 (301) 446-3199

目次

1	C++、Flash、ActionScript とのインタフェース.....	1
1.1	ActionScript C++.....	2
1.1.1		

1 C++、Flash、ActionScript とのインタフェース

Flash® ActionScript™



ActionScript → C++	C++ → ActionScript
FSCommand	Gfx::Movie::Get/SetVariable ActionScript
ExternalInterface	Gfx::Movie::Invoke ActionScript
Direct Access API	
Gfx::FunctionHandler	ActionScript VM
	Gfx::Value

1.1 ActionScript から C++

Scaleform C++ ActionScript 2

FSCommand ExternalInterface

ActionScript API Flash FSCommand

ExternalInterface C++ Scaleform

Scaleform

GfX::Loader GfX::MovieDef GfX::Movie Scaleform

[Scaleform](#)

FSCommand ActionScript fscommand fscommand

ActionScript 2 2 const char C++

fscommand ActionScript fscommand C++

ActionScript *flash.external.ExternalInterface.call* ExternalInterface

ActionScript

C++ ExternalInterface const char

ActionScript GfX::Value ActionScript

ExternalInterface.call C++ ExternalInterface

GfX::Value ActionScript ExternalInterface ActionScript Flash

Player Scaleform External API

FSCommands ExternalInterface

FSCommands ExternalInterface Direct Access API GfX::FunctionHandler

C++

ActionScript VM

C++

ActionScript

GfX::FunctionHandler [Direct Access API](#)

1.1.1 FSCommand コールバック

ActionScript fscommand

ActionScript

fscommand("setMode", "2");

```

ActionScript    fscommand      Gfx FSCommand      2
                Gfx::FSCommandHandler
                fscommand      Gfx::Loader
Gfx::MovieDef      Gfx::Movie
                Gfx::Loader -> Gfx::MovieDef -> Gfx::Movie

                Gfx::Movie      Gfx::RenderConfig
EdgeAA            Gfx::Translator
                Gfx::Loader

                Gfx::Movie
                FSCommand

                GfxPlayerTiny
FxPlayerFSCommandHandler

fscommand      Gfx::FSCommandHandler

```

```
class OurFSCommandHandler : public FSCommandHandler
{
    public:
    virtual void Callback(Movie* pmovie, const char* pcommand,
                        const char* parg)
    {
        Printf("FSCommand: %s, Args: %s", pcommand, parg);
    }
};
```

fscommand

```
// Register our fscommand handler
Ptr<FSCommandHandler> pcommandHandler = *new OurFSCommandHandler;
gfxLoader->SetFSCommandHandler(pcommandHandler);
```

3

1.1.2 外部インタフェース API

Flash ExternalInterface.call

fscommand

ExternalInterface

fscommand

ExternalInterface

```

class OurExternalInterfaceHandler : public ExternalInterface
{
public:
virtual void Callback(Movie* pmovieView, const char* methodName,
                      const Value* args, unsigned argCount)
{
    Printf("ExternalInterface: %s, %d args: ", methodName, argCount);
    for(unsigned i = 0; i < argCount; i++)
    {
        switch(args[i].GetType())
        {
            case Value::VT_Number:
                printf("%3.3f", args[i].GetNumber());
                break;
            case Value::VT_String:
                printf("%s", args[i].GetString());
                break;
        }
        printf("%s", (i == argCount - 1) ? "" : ", ");
    }
    printf("\n");

    // return a value of 100
    Value retValue;
    retValue.SetNumber(100);
    pmovieView->SetExternalInterfaceRetVal(retValue);
}
};

```

GfX::Loader

```
Ptr<GFxExternalInterface> pEIHandler = *new OurExternalInterfaceHandler;
gfxLoader.SetExternalInterface(pEIHandler);
```

ActionScript ExternalInterface

1.1.2.1 FxDelegate

ExternalInterface			
ExternalInterface	methodName		
		methodName	GFx::Value
		FxDelegate	
Apps\Samples\GameDelegate\FxGameDelegate.h			
FxDelegate		GFx::ExternalInterface	FxDelegate
methodName			
	Register/Unregister Handlers		
FxDelegate	minimap	FxDelegate	HUD
	FxDelegate	minimap	

```
// *** Minimap Begin
void FxPlayerApp::Accept(FxDelegateHandler::CallbackProcessor* cbreg)
{
    cbreg->Process("registerMiniMapView", FxPlayerApp::RegisterMiniMapView);

    cbreg->Process("enableSimulation", FxPlayerApp::EnableSimulation);
    cbreg->Process("changeMode", FxPlayerApp::ChangeMode);
    cbreg->Process("captureStats", FxPlayerApp::CaptureStats);

    cbreg->Process("loadSettings", FxPlayerApp::LoadSettings);
    cbreg->Process("numFriendliesChange", FxPlayerApp::NumFriendliesChange);
    cbreg->Process("numEnemiesChange", FxPlayerApp::NumEnemiesChange);
    cbreg->Process("numFlagsChange", FxPlayerApp::NumFlagsChange);
    cbreg->Process("numObjectivesChange", FxPlayerApp::NumObjectivesChange);
    cbreg->Process("numWaypointsChange", FxPlayerApp::NumWaypointsChange);
    cbreg->Process("playerMovementChange", FxPlayerApp::PlayerMovementChange);
```

```

    cbreg->Process("botMovementChange", FxPlayerApp::BotMovementChange);

    cbreg->Process("showingUI", FxPlayerApp::ShowingUI);
}

```

```

FxDelegate
    FxDelegate

```

1.1.2.2 ActionScript から External Interface を使用する

ActionScript External Interface

```

import flash.external.*;
ExternalInterface.call("foo", arg);

```

foo	arg	0	1
		ExternalInterface API	Flash

ExternalInterface

ExternalInterface

```

flash.external.ExternalInterface.call("foo", arg)

```

1.1.2.3 ExternalInterface.addCallback

ExternalInterface.addCallback
C++

ActionScript

this

C++ Gfx::Movie::Invoke()

AS コード

```

import flash.external.*;
var txtField:TextField = this.createTextField("txtField",
                                              this.getNextHighestDepth(), 0, 0, 200, 50);

```

```

var aliasName:String = "setText";
var instance:Object = txtField;
var method:Function = SetText;
var wasSuccessful:Boolean = ExternalInterface.addCallback(aliasName, instance,
                                                         method);

function SetText()
{
    trace(this + ".SetText ");
    this.text = "INVOKED!";
}

```

txtField this

SetText ActionScript

C++コード:

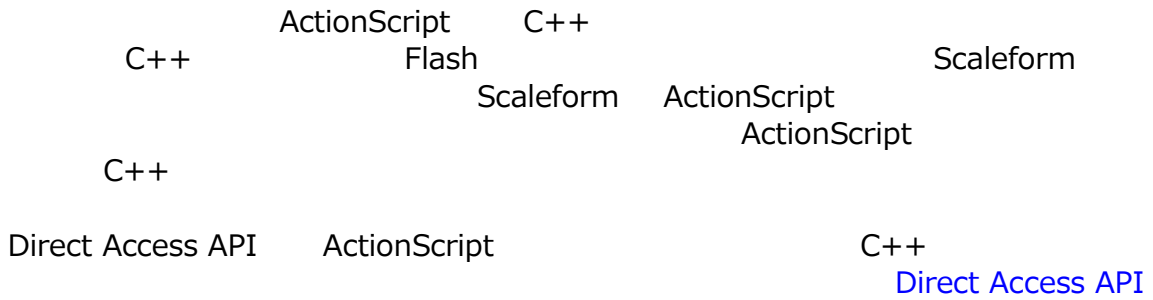
```

pMovie->Invoke("setText", "");
    txtField.SetText
        Gfx::Movie::Invoke()

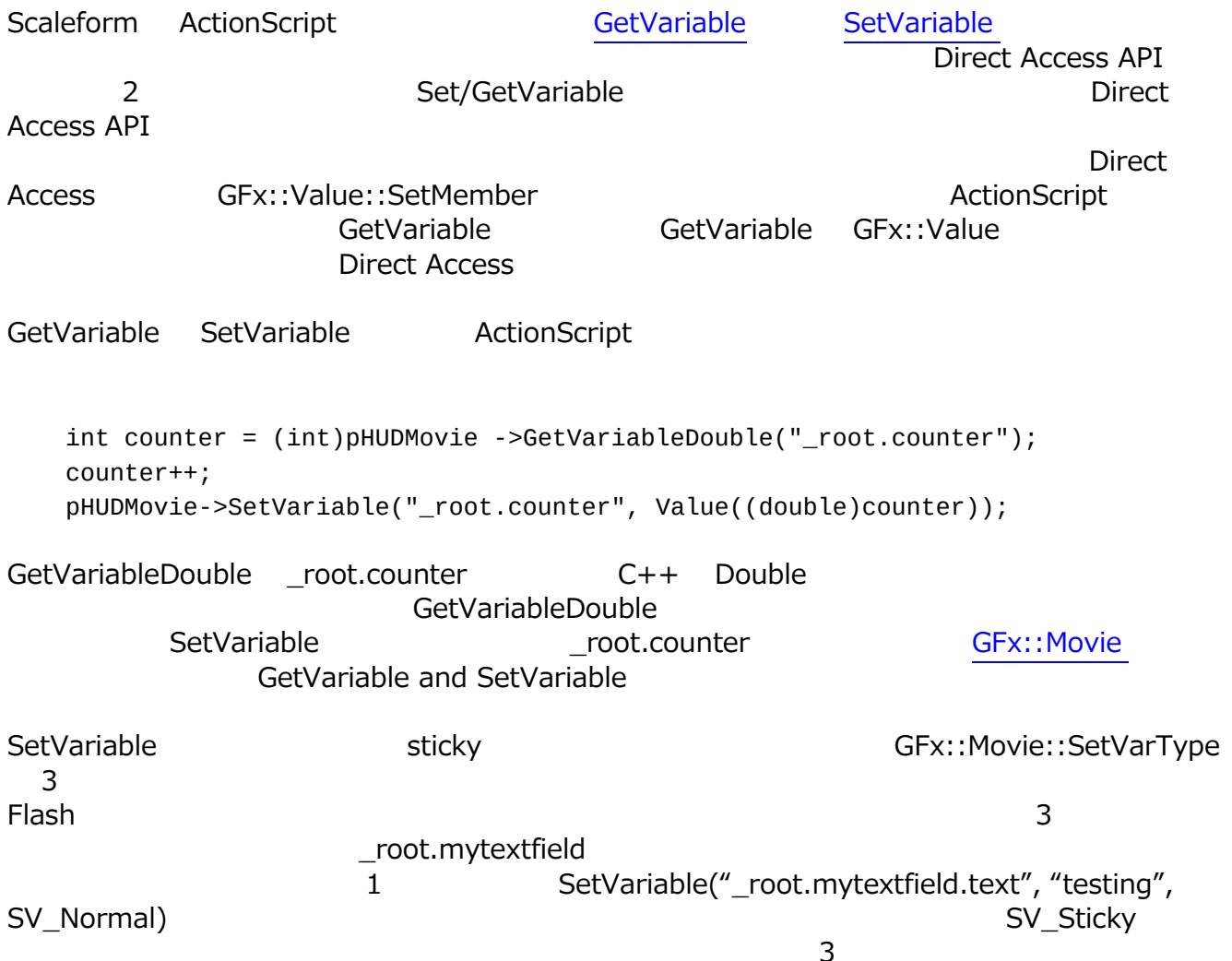
```

INVOKED! txtField
1.2.2

1.2 C++から *ActionScript*



1.2.1 ActionScript 変数の操作



_root.mytextfield.text

SV_ Normal

SV_Normal

C++
SV_Sticky

Scaleform

[SetVariableArray](#) and

[GetVariableArray](#)

SetVariableArray

Gfx::Movie::SetVariableArraySize

SetVariableArray

AS

```
int idxInASArray = 0;
const char* strarr[2];
strarr[0] = "This is the first string";
strarr[1] = "This is the second one";
pMovie->SetVariableArray(Movie::SA_String, "_root.strArray",
                        idxInASArray, strarr, 2);
```

```
int idxInASArray = 0;
const wchar_t* strarr[2];
strarr[0] = "This is the first string";
strarr[1] = "This is the second one";
pMovie->SetVariableArray(Movie::SA_StringW, "_root.strArray",
                        idxInASArray, strarr, 2)
```

GetVariableArray

AS

1.2.2 ActionScript サブルーチンの実行

ActionScript

[Gfx::Movie::Invoke\(\)](#)

ActionScript

UI

UI

Direct Access API

ActionScript

_root

mySlider

SetSliderPos

mySlider

AS コード :

```
this.SetSliderPos = function (pos)
{
    // Clamp the incoming position value
    if (pos<rangeMin) pos = rangeMin;
    if (pos>rangeMax) pos = rangeMax;
    gripClip._x = trackClip._width * ((pos-rangeMin)/(rangeMax-rangeMin));
    gripClip.gripPos = gripClip._x;
}
```

gripClip
rangeMin/Max

mySlider

pos

Invoke

C++コード :

```
Value result;
bool bInvoked = pMovie->Invoke("_root.mySlider.SetSliderPos", &result, "%d",
                                newPos);
```

Invoke

true

false

ActionScript

ActionScript

Invoke

ActionScript

Scaleform

ActionScript

1

ActionScript
true

Gfx::Movie::Advance
Gfx::MovieDef::CreateInstance

initFirstFrame

Invoke

[Gfx::Movie::IsAvailable\(\)](#)

AS

```
Value result;
if (pMovie->IsAvailable("parentPath.mySlider.SetSliderPos"))
    pMovie->Invoke("parentPath.mySlider.SetSliderPos", &result, "%d", newPos);
```

Invoke printf

Invoke

Gfx::Value

```
Value args[3], result;  
args[0].SetNumber(i);  
args[1].SetString("test");  
args[2].SetNumber(3.5);  
pMovie->Invoke("path.to.methodName", &result, args, 3);
```

InvokeArgs

Invoke

Invoke

InvokeArgs

printf

va_list

vprintf

1.2.3パス

Flash

Gfx::Movie

```
Movie::IsAvailable(path)  
Movie::SetVariable(path, value)  
Movie::SetVariableDouble(path, value)  
Movie::SetVariableArray(path, index, data, count)  
Movie::SetVariableArraySize(path, count)  
Movie::GetVariable(value, path)  
Movie::GetVariableDouble( path)  
Movie::GetVariableArray(path, index, data, count)  
Movie::GetVariableArraySize(path)  
Movie::Invoke(pathToMethodName, result, argList, ...)
```

.

clip2

clip1

Main Stage -> clip1 -> clip2

clip1

clip2 clip1

```
"clip1.clip2"
"_root.clip1.clip2"
"_level0.clip1.clip2"
```

```
"Clip1.clip2"  <-          Clip1
"clip2"        <-      Clip1

"_root"  "_level0"      ActionScript 2
                                   ActionScript 3
                                           ActionScript 3
                                           _root  _level0  root  level0

SWF                                _root
                                   _levelN
```

```
Flash Studio  Test Movie  Ctrl-Enter          Ctrl-Alt-V
```

```
Scene 1          Flash Studio
```

```
Ctrl-Alt-V
```

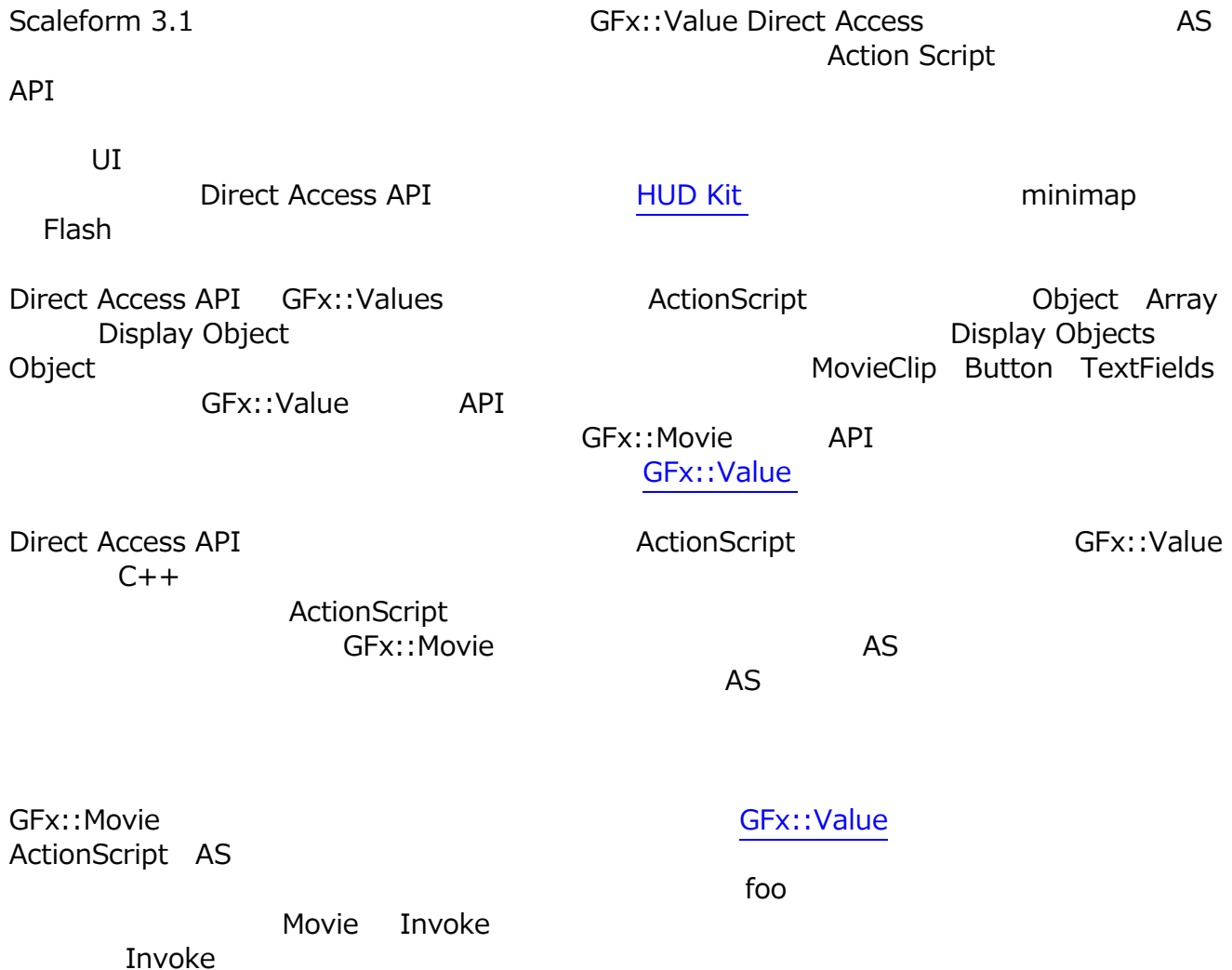
```
loadMovie
    _level2      Level
        Object
_levelN.objectPath.objectName

                                   _level0
                                   _levelN.objectName
                                   N

MovieClip                                _level1
```

1. [Paths to Objects and Variables](#) Jesse Stratford
2. [Advanced Pathing](#) Jesse Stratford

2 Direct Access API



1. Gfx::Movie Invoke
Value ret;
Value args[N];
pMovie->Invoke("_root.foo.method", &ret, args, N);
2. Direct Access Invoke
(Assumes 'foo' holds a reference to an AS object at "_root.foo")
Value ret;
Value args[N];
bool = foo.Invoke("method", &ret, args, N);

Invoke AS
Gfx::Value::HasMember GetMember SetMember Direct Access API

movieClip Gfx::Value GetMember

```
Value tf;
movieClip.GetMember("textField", &tf);
tf.SetText("hello");
```

movieClip textField SetText

2.1 オブジェクトとアレイ

Direct Access API AS

C++ snippet
2

```
Movie* pMovie = ... ;
Value parent, child;
pMovie->CreateObject(&parent);
pMovie->CreateObject(&child);
parent.SetMember("child", child);
```

[CreateObject](#) ActionScript
CreateObject

```
Value mat, params[6];
// (set params[0..6] to matrix data) ...
pMovie->CreateObject(&mat, "flash.geom.Matrix", params, 6);
```

Object Array

```
// (Assuming pMovie is a Gfx::Movie*):
Value owner, childArr, childObj;
pMovie->CreateArray(&owner); // create parent array
pMovie->CreateArray(&childArr); // create child array
pMovie->CreateObject(&childObj); // create child object
bool = owner.SetElement(0, childArr); //set parent[0] to child array
bool = owner.SetElement(1, childObj); // set parent[1]to child object
```

```
...
bool = foo.SetMember("owner", owner); // later, set the 'owner' variable in
                                       // object foo, to the parent object
```

[Gfx::Value::VisitMembers\(\)](#)

Visit

ObjectVisitor

Array DisplayObject

VisitMembers を使うことはできませんので注意し

enumerable

てください。

ActionScript

ASSetPropFlags

```
: _global.ASSetPropFlags(MyClass.prototype, ["someFunc", "__get__number",
                                             "test"], 6, 1);
```

getter/setter

ASSetPropFlags

VisitMembers

enumerable

Direct Access

VT_Object

VT_Object

ASSetPropFlags

enumerable

2.2 ディスプレイオブジェクト

Direct Access API
MovieClip Button

DisplayObject
TextField

Object

[DisplayInfo](#) API

DisplayInfo API
DisplayObject

SetMember

SetDisplayInfo

Gfx::Movie::SetVariable

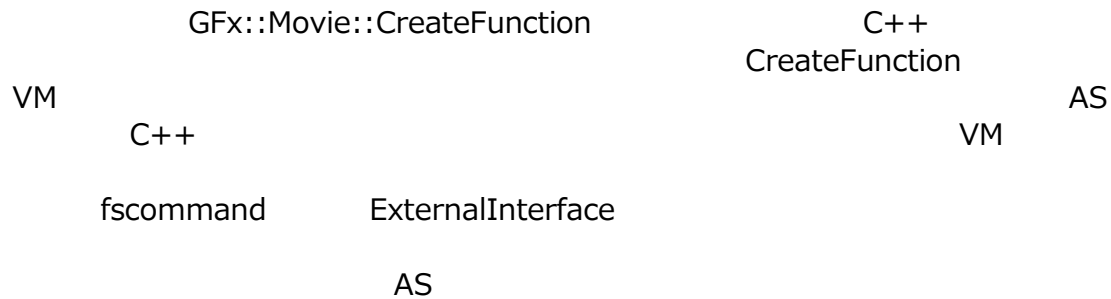
SetDisplayInfo

movieclip

```
// Assumes movieClip is a Gfx::Value*
Value::DisplayInfo info;
PointF pt(100,100);
info.SetRotation(90);
info.SetPosition(pt.x, pt.y);
movieClip.SetDisplayInfo(info);
```

2.3 関数オブジェクト

ActionScript2 Virtual Machine AS2 VM



```

Value obj;
pmovie->GetVariable(&obj, "_root.obj");

class MyFunc : public FunctionHandler
{
    public:
        virtual void Call(const Params& params)
        {
            // Callback logic/handling
        }
};

Ptr<MyFunc> customFunc = *SF_HEAP_NEW(Memory::GetGlobalHeap()) MyFunc();
Value func;
pmovie->CreateFunction(&func, customFunc);
obj.SetMember("func", func);
  
```

ActionScript _root timeline

```
obj.func(param1, param2, param3);
```

Call() Param

- pRetVal: Gfx::Value
pMovie->CreateObject() CreateArray()
 Number Boolean
- pMovie:
- pThis:
- ArgCount:
- pArgs: Gfx::Values []

```
Value firstArg = params.pArgs[0];
```

- pArgsWithThisRef:
- pUserData:

VM

VM

```
Value networkProto, networkCtorFn, networkConnectFn;
class NetworkClass : public FunctionHandler
{
public:
    enum Method
    {
        METHOD_Ctor,
        METHOD_Connet,
    };

    virtual ~NetworkClass() {}
    virtual void Call(const Params& params)
    {
        int method = int(params.pUserData);
        switch (method)
        {
            case METHOD_Ctor:
                // Custom logic
                break;
            case METHOD_Connet:
                // Custom logic
                break;
        }
    }
};

Ptr<NetworkClass> networkClassDef = *SF_HEAP_NEW(Memory::GetGlobalHeap())
                                    NetworkClass();

// Create the constructor function
pmovie->CreateFunction(&networkCtorFn, networkClassDef,
                    (void*)NetworkClass::METHOD_Ctor);
// Create the prototype object
pmovie->CreateObject(&networkProto);
// Set the prototype on the constructor function
networkCtorFn.SetMember("prototype", networkProto);
```

```

// Create the prototype method for 'connect'
pmovie->CreateFunction(&networkConnectFn, networkClassDef,
                    (void*)NetworkClass::METHOD_Connet);
networkProto.SetMember("connect", networkConnectFn);
// Register the constructor function with _global
pmovie->SetVariable("_global.Network", networkCtorFn);

```

VM

```
var netObj:Object = new Network(param1, param2);
```

VM

VM

```

Value origFuncReal;
obj.GetMember("funcReal", &origFuncReal);
class FuncRealIntercept : public FunctionHandler
{
    Value OrigFunc;
public:
    FuncRealIntercept(Value origFunc) : OrigFunc(origFunc) {}
    virtual ~FuncRealIntercept() {}
    virtual void Call(const Params& params)
    {
        // Intercept logic (beginning)
        OrigFunc.Invoke("call", params.pRetVal, params.pArgsWithThisRef,
                        params.ArgCount + 1);
        // Intercept logic (end)
    }
};
Value funcRealIntercept;
Ptr<FuncRealIntercept> ;};

> };

```

l", , 1s Tj /C,;1s Tj34BDC8EMC9.MC Tm

2.4.1 オブジェクトのサポート

説明	パブリックメソッド "_root.foo"	'foo'
	bool has = foo.HasMember("bar");	
	Value bar; bool = foo.GetMember("bar", &bar);	
	Value val; bool = foo.SetMember("bar", val);	
	Value ret; Value args[N]; bool = foo.Invoke("method", args, N, &ret); or foo.Invoke("method", args, N);	
	Value obj; pMovie->CreateObject(&obj); ... bool = foo.SetMember("bar", obj);	
	Value owner, child; pMovie->CreateObject(&owner); pMovie->CreateObject(&child); bool = owner.SetMember("child", child); ... bool = foo.SetMember("owner", owner);	
	Value::ObjectVisitor v; foo.VisitMembers(&v);	
	bool = foo.DeleteMember("bar");	

2.4.2 アレイのサポート

説明	パブリックメソッド "_root.foo.bar"	'foo'	'bar'
	unsigned sz = bar.GetArraySize();		

	Value val; bool = bar. GetElement (idx, &val);
	Value val; bool = bar. SetElement (idx, val);
	bool = bar. SetArraySize (N);
	Value arr; pMovie-> CreateArray (&arr); ... bool = foo. SetMember ("bar", arr);
	Value owner, childArr, childObj; pMovie-> CreateArray (&owner); pMovie-> CreateArray (&childArr); pMovie-> CreateObject (&childObj); bool = owner. SetElement (0, childArr); bool = owner. SetElement (1, childObj); ... bool = foo. SetMember ("owner", owner);
	Enumeration for (UPInt i=0; i <N; i++) { ... bool = bar. GetElement (i+idx, &val) ... } Using a visitor pattern: Value::ArrayVisitor v; bar. VisitElements (v, idx, N);
	bool = bar. ClearElements ();
	PushBack Value val; bool = bar. PushBack (val); PopBack Value val; bool = bar. PopBack (&val); or void bar. PopBack ();
	Single element bool = bar. RemoveElement (idx); Series of elements bool = bar. RemoveElements (idx, N);

2.4.3ディスプレイオブジェクトのサポート

説明	パブリックメソッド "_root.foo.bar" TextField Button	'foo'	MovieClip
----	--	-------	-----------

	Value::DisplayInfo info; bool = foo. GetDisplayInfo (&info);
	Value::DisplayInfo info; ... bool = foo. SetDisplayInfo (info);
	Matrix2_3 mat; bool = foo. GetDisplayMatrix (&mat);
	Matrix2_3 mat; ... bool = foo. SetDisplayMatrix (mat);
	Render::Cxform cxform; bool = foo. GetColorTransform (&cxform);
	Render::Cxform cxform; ... bool = foo. SetColorTransform (cxform);

2.4.4 MovieClip のサポート

説明	パブリックメソッド "_root.foo" MovieClip 'foo'
MovieClip	Value newInstance; bool = foo. AttachMovie (&newInstance, "SymbolName", "instanceName");
MovieClip MovieClip	Value emptyInstance; bool = foo. CreateEmptyMovieClip (&emptyInstance, "instanceName");
keyframe	bool = foo. GotoAndPlay ("myframe");
keyframe	bool = foo. GotoAndPlay (3);
keyframe	bool = foo. GotoAndStop ("myframe");
keyframe	bool = foo. GotoAndStop (3);

2.4.5 Textfield のサポート

説明	パブリックメソッド "_root.foo" Textfield 'foo'
raw textField	Value text; bool = foo. GetText (&text);
raw textField	Value text; ... bool = bar. SetText (text);

HTML	Value htmlText; bool = bar.GetTextHTML (&htmlText);
HTML	Value htmlText; ... bool = bar.SetTextHTML(htmlText);